



Lab 1: Introduction to JAVA.

Objective(s):

1. What is Java?
2. Features and applications of Java.
3. JAVA Basics.
4. Installing JDK.
5. Writing HelloWorld.java in Text Editor
6. JAVA variables & data types.
7. Input & Output
8. Java Variable Type Conversion & Type Casting

1: What is JAVA?

Java was developed by Sun Microsystems in 1995, but it has stood the test of time and remains highly relevant and widely used to this day. Java is a high-level, general-purpose object-oriented programming language.

Java versions follow a release cadence, with major Long-Term Support (LTS) versions like Java 8, 11, 17, 21, and the latest **Java 25 (LTS as of late 2025)** offering extended support, while intermediate versions (9, 10, 12, 13, 14, 15, 16, 18-20, 22-24) provide new features but shorter support cycles. Key versions include **Java 8 (still widely used)**, **Java 11**, **Java 17**, and the current LTS, **Java 21**, with Java 25 being the newest LTS release for enterprise stability and feature adoption, though users need to check Oracle's licensing for different versions.

Key Java LTS Versions:

- **Java 8 (LTS):** Still popular, offers lambdas, type annotations, and free public updates for personal/dev use.
- **Java 11 (LTS):** Introduced local-variable syntax for lambdas.
- **Java 17 (LTS):** Features sealed types and stable floating-point semantics, widely adopted.
- **Java 21 (LTS):** Includes record patterns and pattern matching for switch.

- **Java 25 (LTS):** The newest LTS version as of late 2025, adding features like compact source files.

How Java Versions Work:

- **LTS (Long-Term Support):** These versions receive security updates and bug fixes for several years, making them ideal for large applications and enterprises (e.g., 8, 11, 17, 21, 25).
- **Feature Releases:** Versions between LTS releases (e.g., 9, 10, 12, 13, 14, 15, 16, 18-20, 22-24) introduce new features but are supported for shorter periods.
- **Versioning:** Major releases use numbers like 8, 11, 17, 21, 25, while updates are indicated by update numbers (e.g., 8u202, 17.0.7).

Where to Get Java:

- **Oracle:** Provides downloads for various JDKs (Java Development Kit).
- **OpenJDK:** The open-source foundation, with builds from various providers.

Choosing a Version:

- For new projects, **Java 21 or 25 (LTS)** is recommended for stability and modern features.
- **Java 8** remains relevant for legacy systems needing free updates for personal use.

Java Platforms

According to Oracle, there are four platforms of the Java programming language

- Java Platform, Standard Edition (Java SE)
- Java Platform, Enterprise Edition (Java EE)
- Java Platform, Micro Edition (Java ME)
- JavaFX

2: Features and applications of JAVA

Features of Java

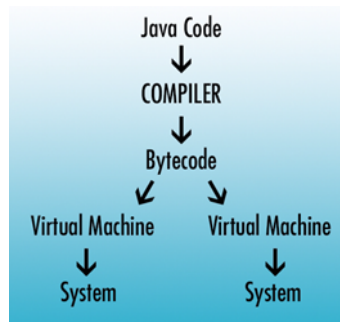
The following are Java's main features:

- **High level and general purpose:** Rather than being created to accomplish one very specific task, Java allows us to write computer-readable instructions in an open-ended environment. Because it's not really feasible, or even desirable, for every computer system to have its own specialized programming language, the vast majority of the code is written in high-level, general-purpose languages such as Java.

Note: Compiling a programming language such as JAVA is the act of taking the human-readable code that we've written and converting it into an interpreted machine-friendly code. Unfortunately, it's usually not very friendly for humans to read or write. To do this, we use a program called the compiler that takes in our code as text and converts it into machine code.

- **Object-oriented:** Java is also what we call an object-oriented language. While we won't get into the specifics of objects and classes until a bit later in this book, know for now that objects allow us to define **modular entities within our program that make them much more human-readable and much more manageable to create large-scale software projects**. A firm grasp of object-oriented concepts is absolutely essential for any modern software developer.
- **Platform-independent:** Lastly, Java was designed with the intention that it be a write once, run anywhere language. This means if you and I both have systems with Java installed and even if our systems are not normally identical--for example, I'm on a Windows machine and you're on a Mac--a Java program on my machine that I give to you will still run essentially the same on your machine without the need for it to be recompiled.

Traditionally, we would have system that it was going to run different idea of what their Java circumvents this issue by same type of interpreted code



to recompile a program for every on because all systems have a machine code should look like. compiling all Java programs to the called bytecode.

A compiled Java program in bytecode can be run by any system in which Java is installed. This is because when we install Java on your system, we also install a Java virtual machine with it that's specific to that system. It is this machine's responsibility to convert the bytecode into the final instructions that head to the processor in that system.

By making it the system's responsibility to do this final conversion, Java has created a **write once, run anywhere** language where I can hand you a Java program and you can run it on your machine while being fairly certain that it's going to run in the same manner that it did on mine. This impressive level of cross-platform support on a language as powerful as Java has made it one of the software developing world's go-to tools for quite some time.

"Write once, run everywhere"

JAVA Applications:

In today's modern times, Java is used to develop desktop applications, web servers, and client-side web applications. It's the native language of the Android operating system, which operates on Android phones and tablets.

Java has been used to write video games and is sometimes even ported to smaller devices without a traditional operating system. It remains a huge player in today's technical world.

3: JAVA Basics

JDK stand for **Java Development Kit**, is a software development environment used for developing Java applications and applets. It includes the Java Runtime Environment (JRE), an interpreter/loader (Java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc) and other tools needed in Java development.

JRE stands for **Java Runtime Environment**. The Java Runtime Environment provides the minimum requirements for executing a Java application; it consists of the *Java Virtual Machine (JVM)*, *core classes*, and *supporting files*.

JVM: A Java Virtual Machine (JVM), an implementation of the Java Virtual Machine Specification, interprets compiled Java binary code (called bytecode) for a computer's processor (or "hardware platform") so that it can perform a Java program's instructions. Java was designed to allow application programs to be built that could be run on any platform without having to be rewritten or recompiled by the programmer for each separate platform.

JIT Just-in-time Compiler is the part of the Java Virtual Machine (JVM) that is used to speed up the execution time. JIT interpret parts of the bytecode that have similar functionality at the same time, and hence reduces the amount of time needed for full interpretation.

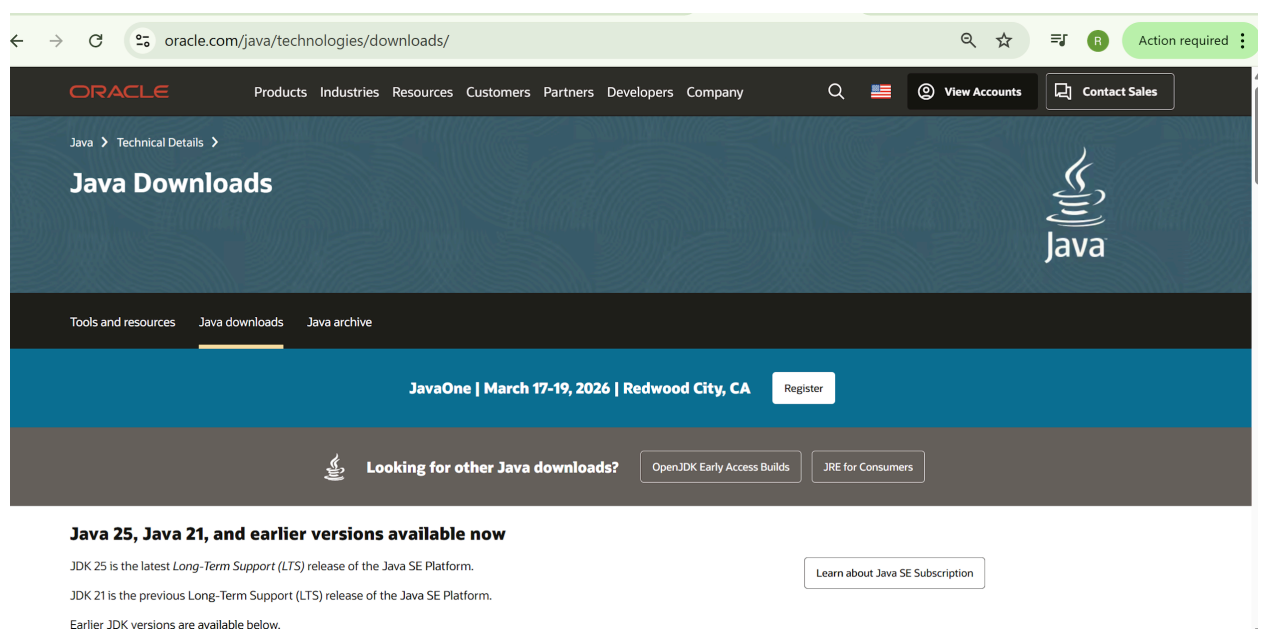
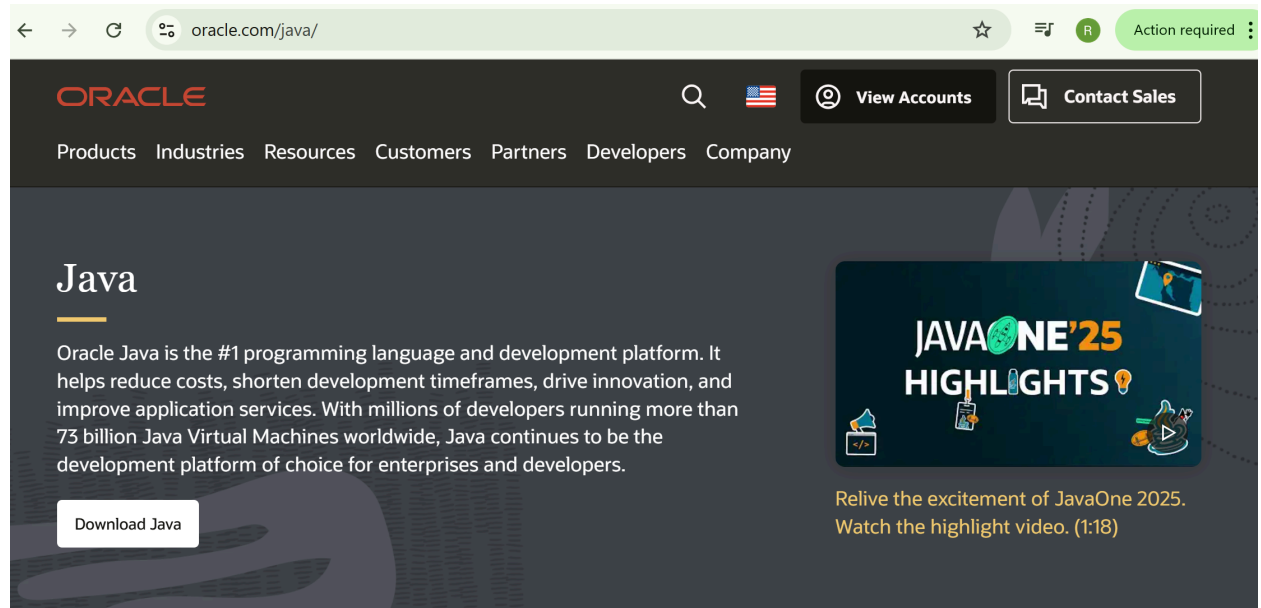
4: Installing JDK

Java Development Kit

To develop Java applications on our computers, we require a JDK. Visit the link below to download the JDK setup.

Step 1: Download Java (JDK)

1. Go to the official Oracle Java download page (oracle.com/java).
2. Select the latest JDK version (e.g., JDK 25) and choose the installer for your operating system (Windows, macOS, Linux).
3. For Windows, download the .exe (MSI) installer for your system (e.g., x64).



oracle.com/java/technologies/downloads/#jdk25-windows

ORACLE Products Industries Resources Customers Partners Developers Company View Accounts Contact Sales

Tools and resources Java downloads Java archive

JDK 25 JDK 21

Java SE Development Kit 25.0.1 downloads

JDK 25 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions \(NFTC\)](#).

JDK 25 will receive updates under the NFTC, until September 2028, a year after the release of the next LTS. Subsequent JDK 25 updates will be licensed under the [Java SE OTN License \(OTN\)](#) and production use beyond the [limited free grants](#) of the OTN license will [require a fee](#).

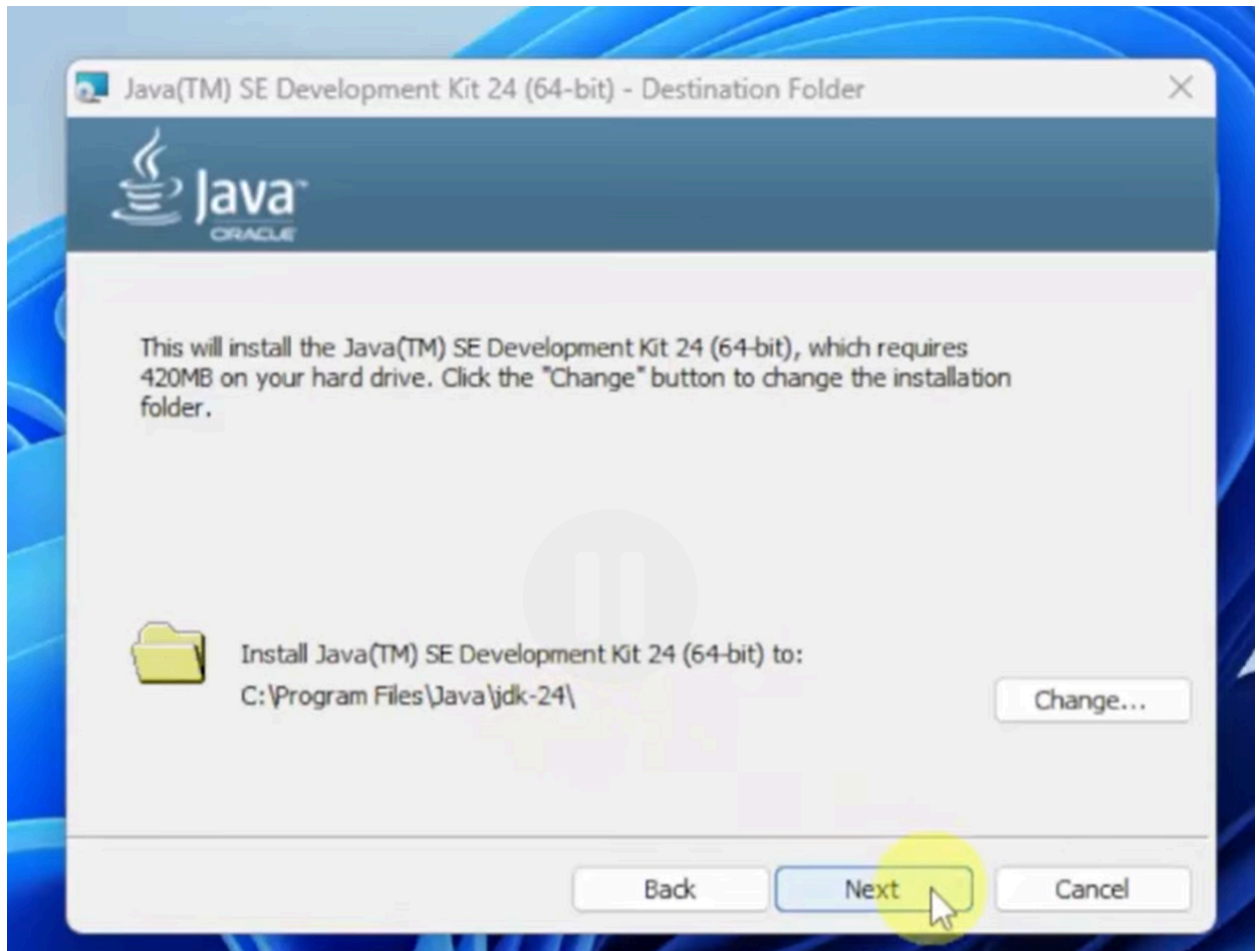
Linux macOS **Windows**

Product/file description	File size	Download
x64 Compressed Archive	204.33 MB	https://download.oracle.com/java/25/latest/jdk-25_windows-x64_bin.zip (sha256)
x64 Installer	182.29 MB	https://download.oracle.com/java/25/latest/jdk-25_windows-x64_bin.exe (sha256)
x64 MSI Installer	181.05 MB	https://download.oracle.com/java/25/latest/jdk-25_windows-x64_bin.msi (sha256)

Documentation Download

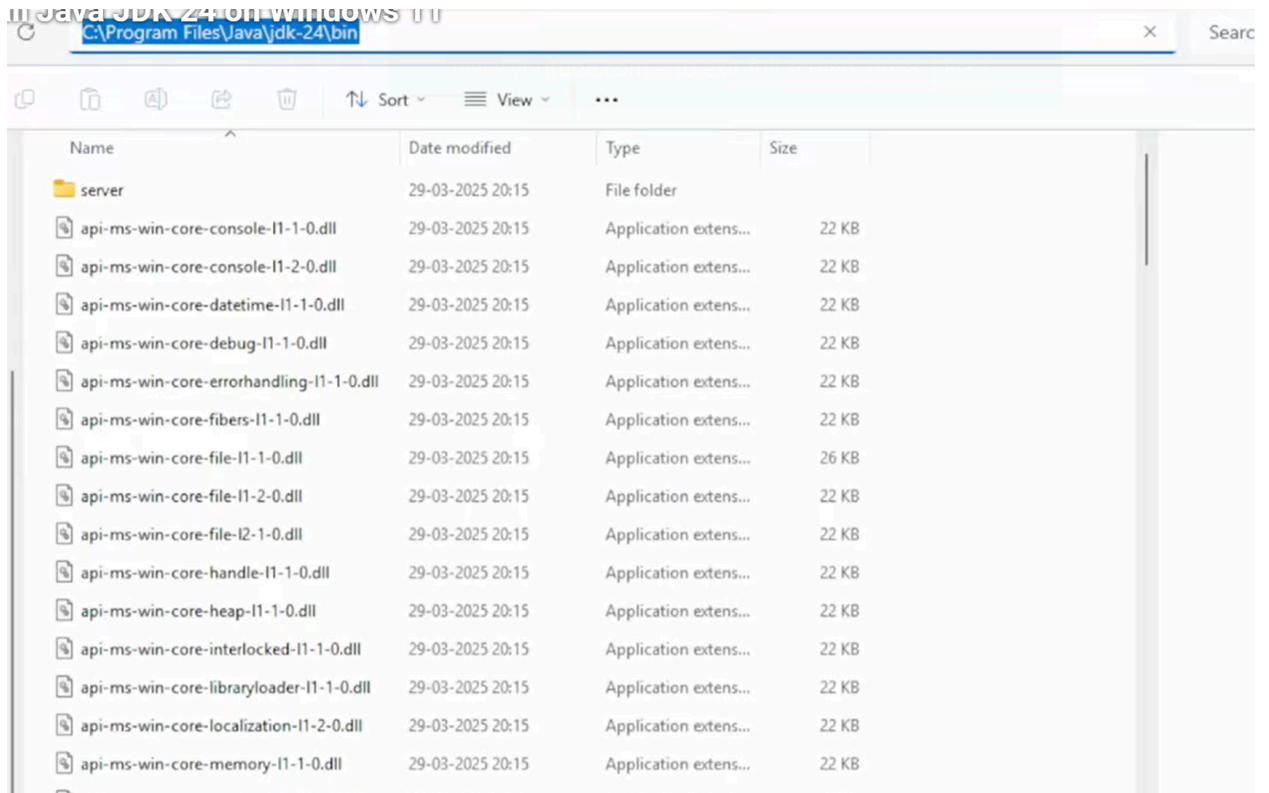
Step 2: Run the Installer

1. Locate the downloaded `.exe` file and double-click it to start.
2. Click **Next**, accept the license agreement, and follow the on-screen instructions, keeping the default installation path (e.g., `C:\Program Files\Java\jdk-XX`).
3. Click **Close** when the installation finishes.

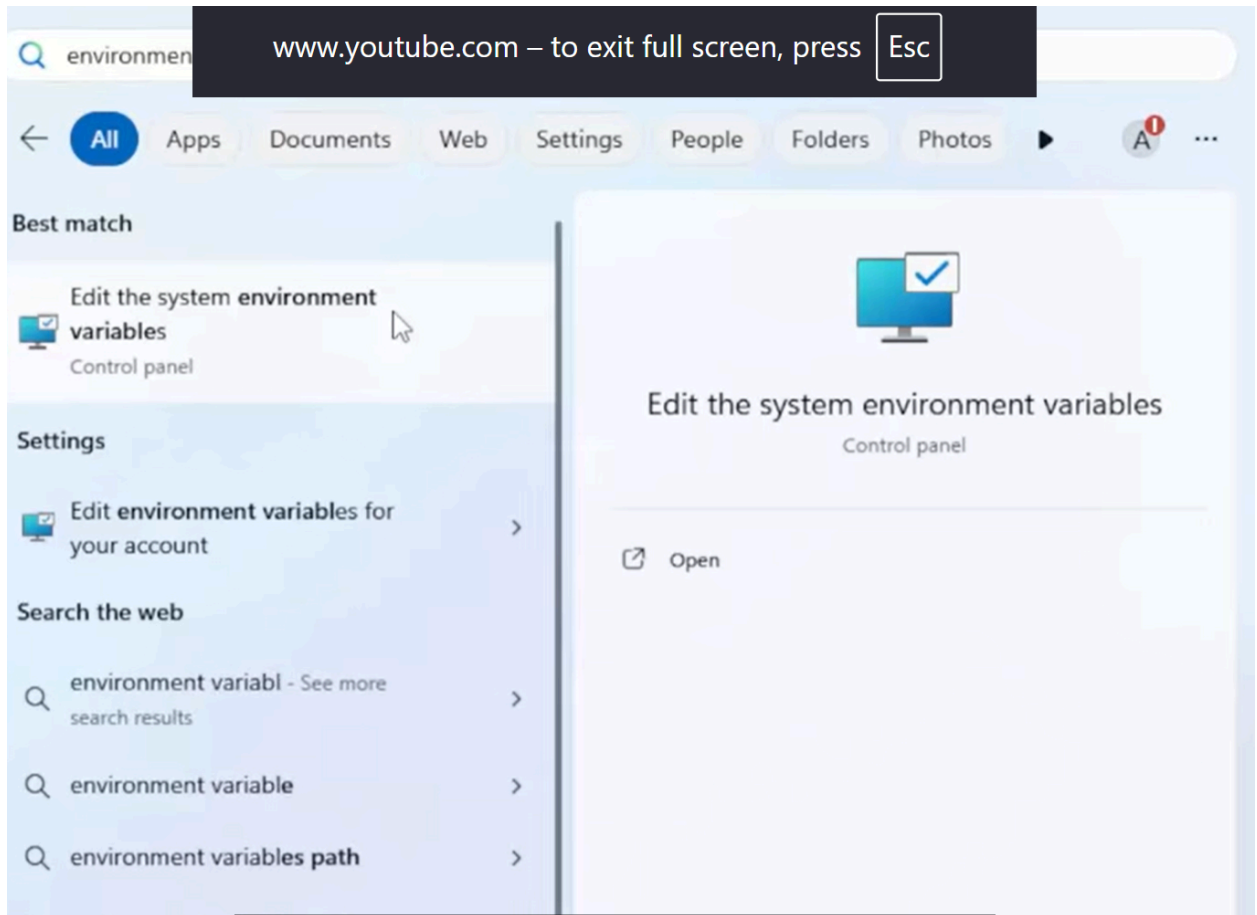


Step 3: Set Environment Variables (Windows)

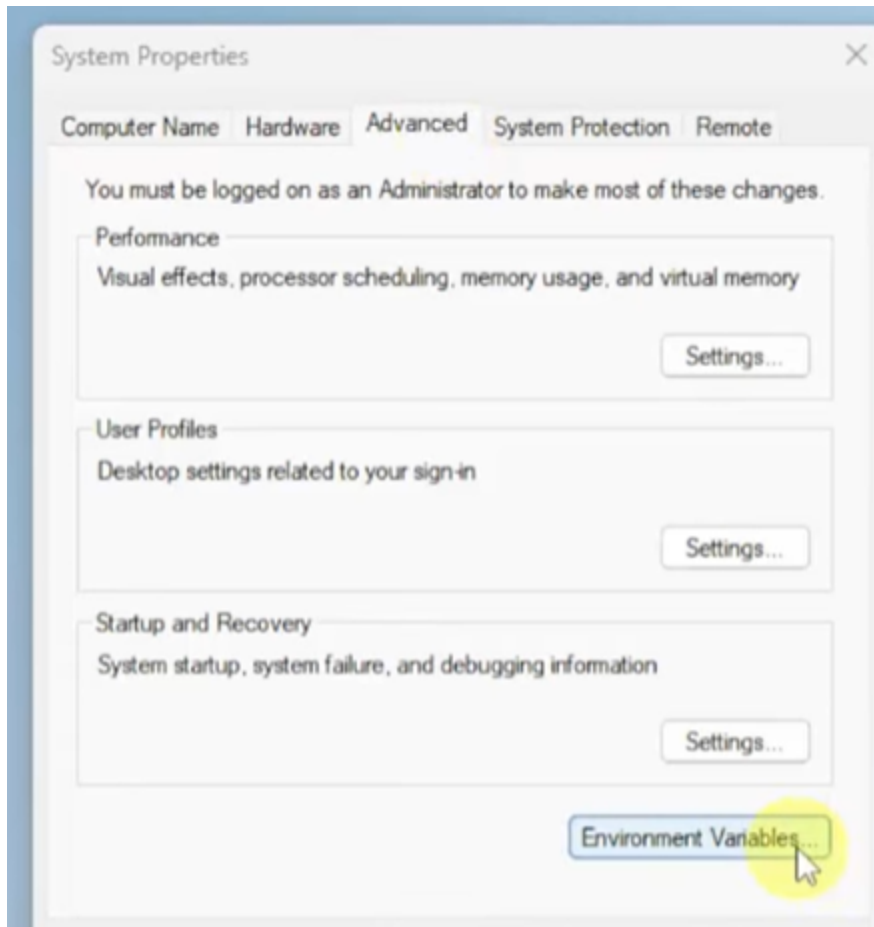
1. Find your installed JDK's bin folder path (e.g., C:\Program Files\Java\jdk-XX\bin) and copy it.



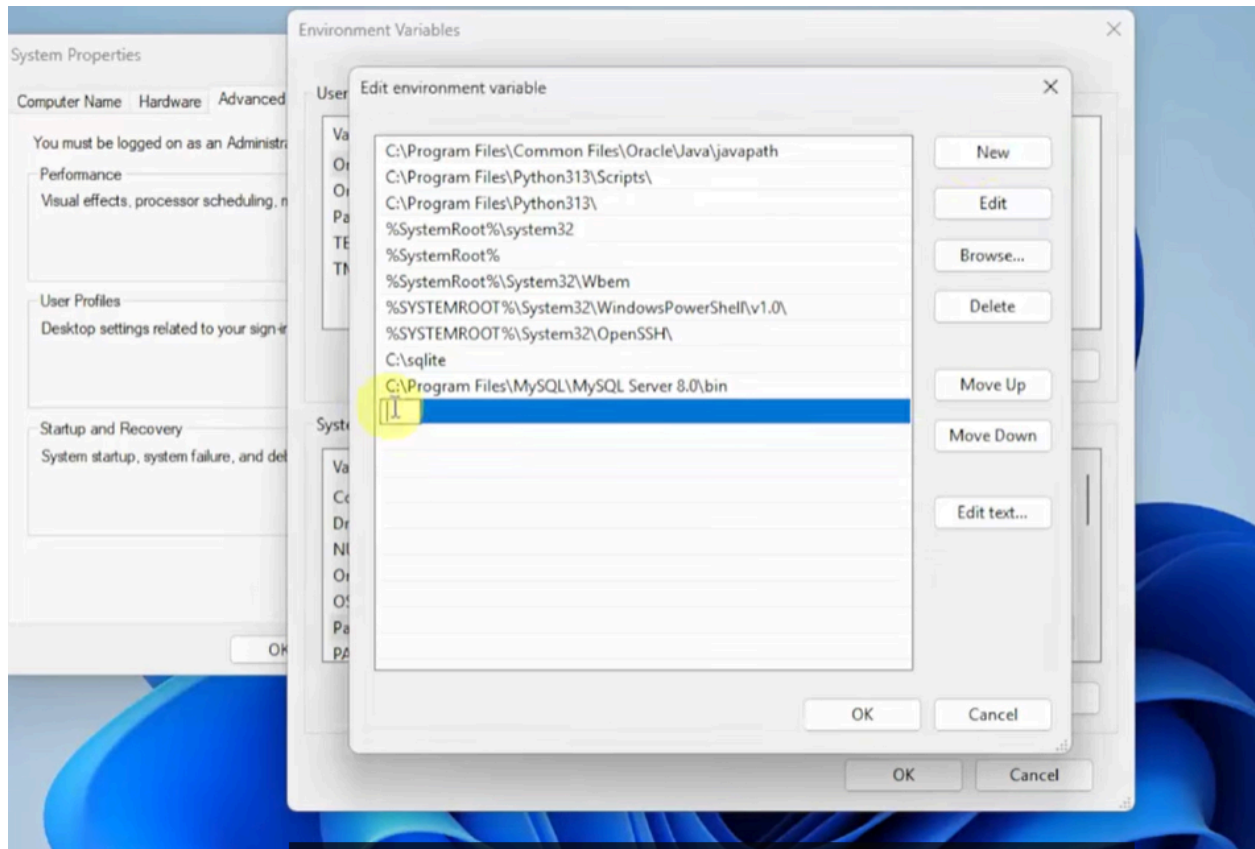
2. Search for "Edit the system environment variables" and open it.
3. In System Properties, click **Environment Variables**.



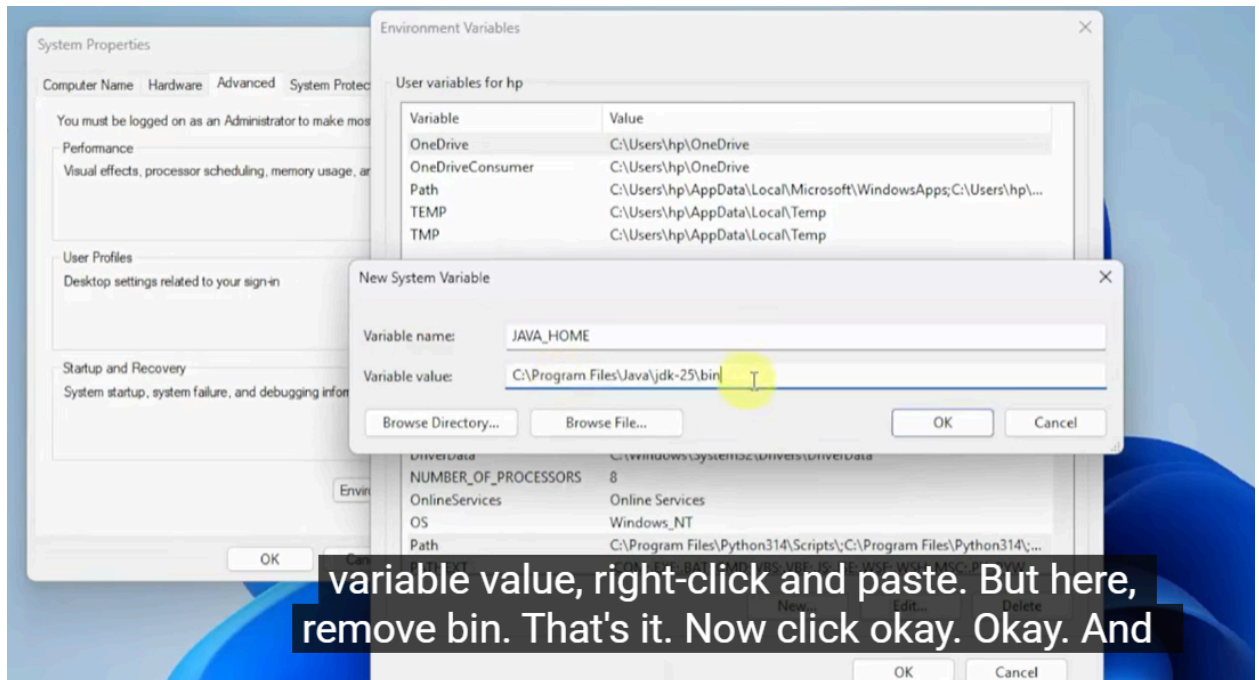
- 4.
5. Under **System variables**, find and select the **Path** variable, then click **Edit**.



6.



7. Click **New**, paste the copied bin folder path, and click **OK**.
8. Click **New** again, set the Variable name to `JAVA_HOME`, and paste the path to the JDK folder (e.g., `C:\Program Files\Java\jdk-XX`), *without* the `\bin` part.



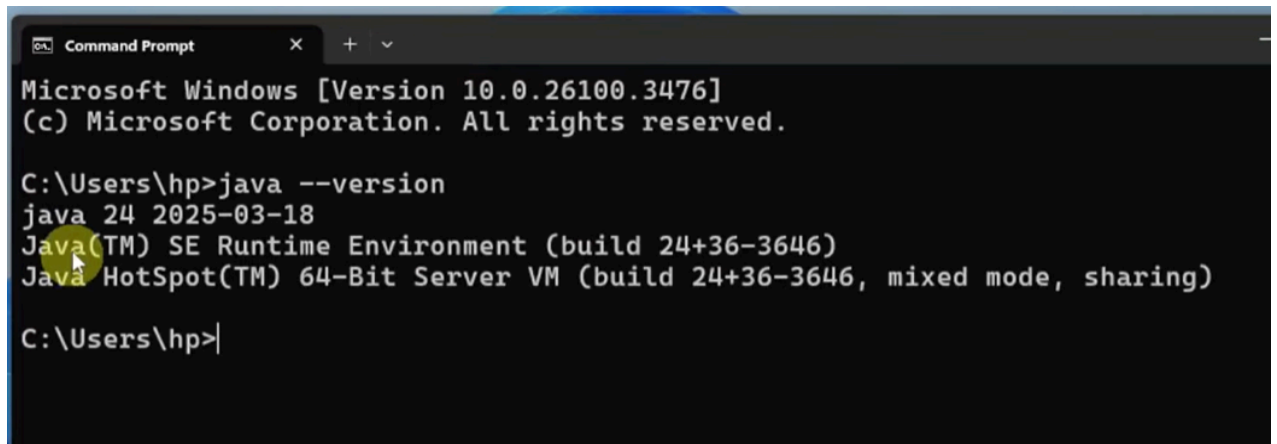
9. Click **OK** on all windows to close them.

Step 4: Verify Installation

1. Open Command Prompt (CMD).
2. Type `java -version` and press Enter.
3. If installed correctly, the command prompt will display the installed Java version, confirming it's working

Why to set the path Variable?

The **PATH environment variable** is set when installing Java so that your operating system can locate the Java executable files (like `java` and `javac`) from any command line or terminal window, without you having to type the full directory path every time.



```
Microsoft Windows [Version 10.0.26100.3476]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hp>java --version
java 24 2025-03-18
Java(TM) SE Runtime Environment (build 24+36-3646)
Java HotSpot(TM) 64-Bit Server VM (build 24+36-3646, mixed mode, sharing)

C:\Users\hp>
```

5: Writing HelloWorld.java in Text Editor

1. **Create a source file** - A source file contains code, written in the Java programming language, that you and other programmers can understand. You can use any text editor to create and edit source files. Run notepad and enter your code. Save this file with the .java extension.

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!"); // Prints
the message to the console
    }
}
```

2. **Compile the source file into a .class file** - Open Command Prompt and enter **javac filename.java**. The Java programming language *compiler* (javac) takes your source file and translates its text into instructions known as *bytecodes*. Next, enter **java ClassName**. The Java application *launcher tool* (java) uses the Java virtual machine to run your application.

6: JAVA Variables & Data Types

Types of variables

In Java, there are three types of variables:

1. Local Variables

2. Instance Variables
3. Static Variables

1. Local Variables

Local Variables are a variable that are declared inside the body of a method.

2. Instance Variables

Instance variables are defined without the STATIC keyword .They are defined outside a method declaration. They are Object specific and are known as instance variables.

3. Static Variables

Static variables are initialized only once, at the start of the program execution. These variables should be initialized first, before the initialization of any instance variables.

Example: Types of Variables in Java

```
class Variables {  
    int data = 99; //instance variable  
    static int a = 1; //static variable  
    void method() {  
        int b = 90; //local variable  
    }  
}
```

Data Types in Java

Data types classify the different values to be stored in the variable. In java, there are two types of data types:

1. Primitive Data Types
2. Non-primitive Data Types

Primitive Data Types

Primitive Data Types are predefined and available within the Java language. Primitive values do not share state with other primitive values.

There are 8 primitive types: byte, short, int, long, char, float, double, and boolean

Integer data types

```
byte (1 byte)  
short (2 bytes)  
int (4 bytes)  
long (8 bytes)
```

Floating Data Type

```
float (4 bytes)
```

double (8 bytes)

Textual Data Type

char (2 bytes)

Logical

boolean (1 bit) (true/false)

7: Input & Output

Output in Java Syntax:

```
System.out.println("Hello World");
```

Input in Java Syntax:

```
// import library
```

```
import java.util.Scanner;
```

```
// Creating scanner object
```

```
Scanner obj = new Scanner(System.in);
```

Taking input from user

```
int num = obj.nextInt(); // Integer Input
```

```
double num = obj.nextDouble(); // Double Input
```

To be covered in next lab.

Lab 8: Java Variable Type Conversion & Type Casting

A variable of one type can receive the value of another type. Here there are 2 cases.

Case 1) Variable of smaller capacity is be assigned to another variable of bigger capacity.

```
double d ;  
int i = 10;  
d = i;
```

This process is Automatic, and non-explicit is known as *Conversion*

Case 2) Variable of larger capacity is be assigned to another variable of smaller capacity.

```
double d = 10;
```

```
int i;
```

```
i = (int) d
```

Type Cast
Operator

In such cases, you have to explicitly specify the **type cast operator**. This process is known as **Type Casting**.

In case, you do not specify a type cast operator; the compiler gives an error. Since this rule is enforced by the compiler, it makes the programmer aware that the conversion he is about to do may cause some loss in data and prevents **accidental losses**.

Example: To Understand Type Casting

```
class Demo {  
    public static void main(String args[]) {  
        byte x;  
        int a = 270;  
        double b = 128.128;  
        System.out.println("int converted to byte");  
        x = (byte) a;  
        System.out.println("a and x " + a + " " + x);  
        System.out.println("double converted to int");  
        a = (int) b;  
        System.out.println("b and a " + b + " " + a);  
        System.out.println("\ndouble converted to byte");  
        x = (byte)b;  
        System.out.println("b and x " + b + " " + x);  
    }  
}
```

Output:

```
int converted to byte  
a and x 270 14  
double converted to int  
b and a 128.128 128  
  
double converted to byte  
b and x 128.128 -128
```

9: if/else, loops

If/Else in Java Syntax

```

if(condition1)
{
    //code to be executed if condition1 is true
}
else if(condition2)
{
    //code to be executed if condition2 is true
}
else if(condition3)
{
    //code to be executed if condition3 is true
}
...
else
{
    //code to be executed if all conditions are false
}

```

For Loop Syntax in Java

```

for(initialization;condition;incr/decr){
    //statement or code to be executed
}

```

Lab Tasks:

Exercise 1 (JAVA Environment Installation & Error Messages)

1. Set up a Java development environment. In the *main()* method of your program try to compile the following invalid Java code snippets. Record the error messages you receive. What do you think each error message indicates?

```
System.out.println("Hello World")
```

```
System.out.println(Hello World)
```

```
System.out.println"Hello World";
```

```
println("Hello World");
```

2. To generate one final error message, remove one of the brackets from the end of your program. Now what message do you receive?

Exercise 2 (Mathematical Expressions)

Write Java code for following mathematical expressions to compute the result.

$f = 5 * 7 / 3$

`float f = (5 * 7) / 3.0f`

`int x = 5 + 'a'`

`int x = 2147483647 + 1`

Exercise 3 (Java Variables)

Write a Java code in which create all of the primitives (except long and double) with different values. Concatenate them into a string and print it to the screen so it will print:

H3110 w0r1d 2.0 true

Exercise 4 (Operators)

1. Write Java program to allow the user to input two integer values and then the program prints the results of adding, subtracting, multiplying, and dividing among the two values. See the example below:

Enter value a:30

Enter value b:10

The result of adding is 40.

The result of subtracting is 20;

The result of multiplying is 300.

The result of dividing is 3.

2. Change the following program to use compound assignments:

```
class ArithmeticDemo {
```

```

public static void main (String[] args){
    int result = 1 + 2; // result is now 3
    System.out.println(result);

    result = result - 1; // result is now 2
    System.out.println(result);

    result = result * 2; // result is now 4
    System.out.println(result);

    result = result / 2; // result is now 2
    System.out.println(result);

    result = result + 8; // result is now 10
    result = result % 7; // result is now 3
    System.out.println(result);
}
}

```

Exercise 5 (if/else & loops)

- Given Boolean variables A, B, and C, write if statements to check for the following conditions:
 - A and B, but not C
 - Either A, B, or C
 - All or none of A, B, and C
 - A and B
- Using a while loop, write Java code which will multiply all the integers from 1 to a given value. Now, utilize a for loop to generate the same functionality.